

## Table of contents

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>SMT Solving</b>                                     | <b>1</b> |
| Introduction . . . . .                                 | 1        |
| SAT-Solving . . . . .                                  | 2        |
| Le problème SAT . . . . .                              | 2        |
| Satisfaction d'une formule PL . . . . .                | 2        |
| Le problème SAT . . . . .                              | 3        |
| Une méthode ancienne : transformation en CNF . . . . . | 3        |
| Algorithme DPLL (1962) . . . . .                       | 4        |
| Améliorations modernes : CDCL (1996-1999) . . . . .    | 5        |
| Logique du premier ordre et théories . . . . .         | 7        |
| Syntaxe FOL . . . . .                                  | 7        |
| Théories . . . . .                                     | 7        |
| SMT Solving : combiner SAT et théories . . . . .       | 8        |
| Algorithme DPLL(T) . . . . .                           | 10       |
| Exemple complet avec la théorie EUF . . . . .          | 11       |
| Formules FOL avec quantificateurs . . . . .            | 12       |
| Définitions . . . . .                                  | 12       |
| Théorème de Herbrand (1930) . . . . .                  | 12       |
| Exemple . . . . .                                      | 12       |
| Transformation en formule universelle close . . . . .  | 14       |
| Conclusion . . . . .                                   | 14       |

## SMT Solving

### Introduction

SMT (Satisfiability Modulo Theories) étend le problème SAT à des formules logiques incluant des théories spécifiques (arithmétique, tableaux, etc.). Par exemple, la formule :

$$n > 0 \implies n - 1 \leq n$$

Ou encore, dans un contexte de vérification de programme :

$$\neg(aux > 0) \wedge (res + aux = a + b) \wedge aux \geq 0$$

Des outils comme Why3 génèrent de telles formules (via le calcul des plus faibles préconditions, WP) et délèguent leur vérification à un solveur SMT.

Par exemple, pour une postcondition  $P(a, b)$  :

$$\underbrace{b \geq 0 \implies a + b = a + b \wedge b \geq 0}_{P(a,b)}$$

On cherche à prouver :  $\forall a, b, P(a, b)$ .

Nous allons explorer :

- Les mécanismes de base des solveurs SMT.
- Leur intégration dans des outils comme Why3.

## SAT-Solving

### Le problème SAT

La logique propositionnelle manipule des variables booléennes :  $Var = \{p, q, \dots\}$ , pouvant être *true* ou *false*, et des connecteurs logiques :  $\wedge, \vee, \implies, \neg$ .

Une formule PL est définie par la grammaire :

- $F \mapsto P$  (variable)
- $F \mapsto \neg F$
- $F \mapsto F \wedge F \mid F \vee F \mid F \implies F$

**Note** : L'implication est associative à droite :  $p \implies q \implies t \iff p \implies (q \implies t)$ .

### Satisfaction d'une formule PL

Une **interprétation**  $I$  est une fonction  $I : Var \rightarrow \{true, false\}$ .

On étend  $I$  à toute formule PL récursivement :

- Si  $f = p \in Var$ , alors  $I(f) = I(p)$ .
- Si  $f = \neg f'$ , alors  $I(f) = true$  ssi  $I(f') = false$ .
- Si  $f = f_1 \vee f_2$ , alors  $I(f) = true$  ssi  $I(f_1) = true$  ou  $I(f_2) = true$ .
- Si  $f = f_1 \wedge f_2$ , alors  $I(f) = true$  ssi  $I(f_1) = true$  et  $I(f_2) = true$ .
- Si  $f = f_1 \implies f_2$ , alors  $I(f) = true$  ssi si  $I(f_1) = true$  alors  $I(f_2) = true$  (sinon *true* si  $I(f_1) = false$ ).

On dit que :

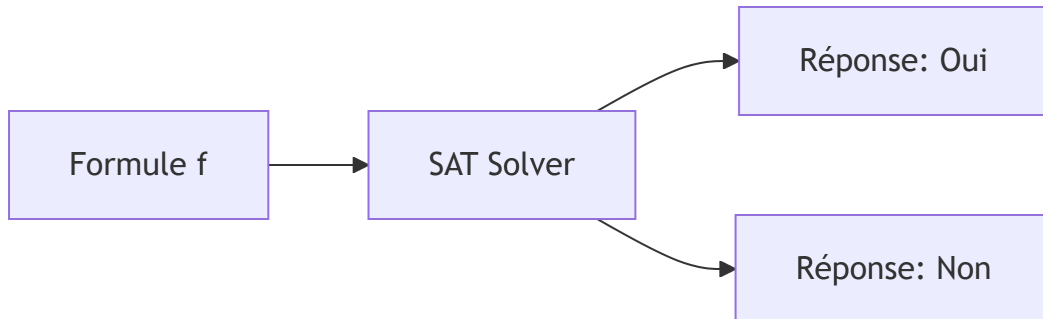
- $I$  **satisfait**  $f$  (noté  $I \models f$ ) ssi  $I(f) = true$ .
- $f$  est **satisfiable** ssi  $\exists I, I \models f$ .

- $f$  est **valide** (une tautologie) ssi  $\forall I, I \models f$ .

Pour prouver qu'une formule  $g$  est valide, on peut utiliser une **preuve par réfutation** : montrer que  $\neg g$  est insatisfiable. Souvent,  $g$  est de la forme *hypothèse*  $\implies$  *conclusion*, donc  $\neg g = \text{hypothèse} \wedge \neg \text{conclusion}$ .

## Le problème SAT

Étant donnée une formule PL  $f$ , le problème SAT consiste à décider si  $f$  est satisfiable.



- Il y a  $2^n$  interprétations possibles pour  $n$  variables.
- Le problème est **décidable** (il existe des algorithmes) mais **NP-complet**, donc aucun algorithme efficace (polynomial) n'est connu en général.
- En pratique, les solveurs peuvent échouer par manque de ressources (temps, mémoire).

## Une méthode ancienne : transformation en CNF

Une **forme normale conjonctive (CNF)** est une conjonction de clauses, où chaque clause est une disjonction de littéraux (une variable ou sa négation).

### Définition : Formule CNF équivalente

Une formule est en CNF si elle est de la forme :

$$C_1 \wedge C_2 \wedge \dots \wedge C_n$$

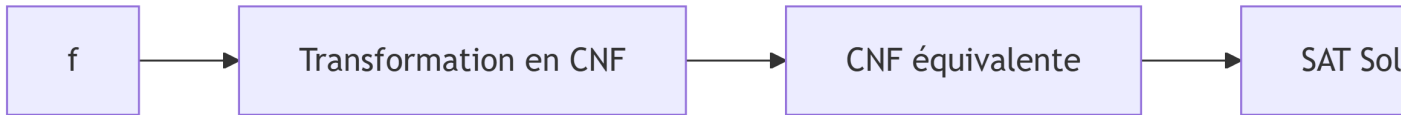
où chaque clause  $C_i$  est de la forme :

$$l_{i1} \vee l_{i2} \vee \dots \vee l_{im}$$

et chaque littéral  $l_{ij}$  est soit une variable  $p \in Var$ , soit sa négation  $\neg p$ .

**Exemple :**  $(p \vee \neg q) \wedge (r \vee s) \wedge \neg p$

On admet que pour toute formule PL  $f$ , il existe une formule PL en CNF qui lui est logiquement équivalente. On peut donc restreindre le problème SAT aux formules CNF.



**Remarques importantes :**

- Simplification : une clause contenant à la fois un littéral et sa négation (ex:  $a \vee \neg a \vee \dots$ ) est toujours vraie et peut être supprimée.
- **Clause unitaire** : une clause contenant un seul littéral (ex:  $a$ ) force la valeur de cette variable. Si  $P \wedge a \wedge Q$  est vraie, alors  $a = \text{true}$  et on peut simplifier en résolvant  $P \wedge Q$ .

### Algorithme DPLL (1962)

Soit  $P$  une formule PL en CNF. Pour un littéral  $x$  (variable ou sa négation), on note  $P[x]$  la formule obtenue en imposant  $x = \text{true}$  et en simplifiant.

Les règles de simplification sont :

- Si  $x$  apparaît dans une clause, cette clause est supprimée (car satisfaite).
- Si  $\neg x$  apparaît dans une clause, on retire  $\neg x$  de cette clause.
- Si une clause devient vide, elle est équivalente à  $\text{false}$ .
- Si une clause unitaire  $\neg x$  reste, alors  $x$  doit être  $\text{false}$ , donc  $P[x] = \text{false}$ .

**Algorithme DPLL(P) :**

```
Si P = true, retourner true.  
Si P = false, retourner false.  
Si P contient une clause unitaire x, retourner DPLL(P[x]).  
Choisir une variable x dans P.  
Retourner DPLL(P[x]) ou DPLL(P[¬x]).
```

**Terminaison** : Le variant est le nombre de variables (diminue de 1 à chaque appel).

**Correction** : Retourne true ssi  $P$  est satisfiable.

**Exemple** :

$$P = (p \vee q \vee r) \wedge (\neg q \vee r) \wedge \neg r \wedge (\neg p \vee q \vee r)$$

- DPLL(P) :
  - $P[\neg r] = (p \vee q) \wedge \neg q \wedge (\neg p \vee q)$
  - $P[\neg r, \neg q] = p \wedge \neg p$
  - $P[\neg r, \neg q, p] = false$
- Donc  $P$  est **insatisfiable**.

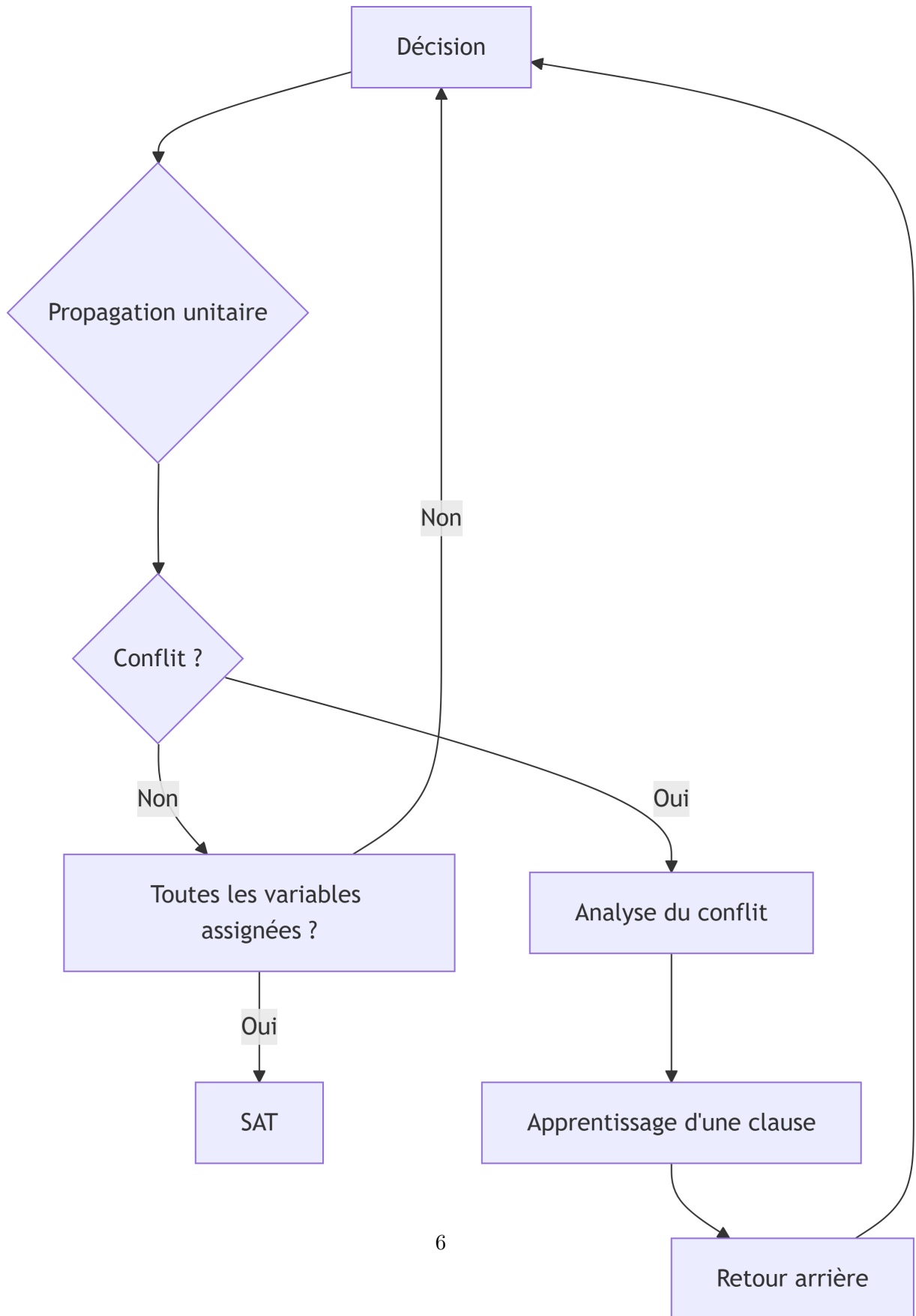
Si DPLL retourne true, on peut reconstruire un modèle (une affectation des variables) en suivant les choix.

### **Améliorations modernes : CDCL (1996-1999)**

L'algorithme **Conflict-Driven Clause Learning (CDCL)** étend DPLL avec :

- L'apprentissage de clauses à partir des conflits.
- Le retour arrière non-chronologique.
- Une heuristique de décision.

Cela permet de traiter des problèmes avec des centaines de milliers de variables et des millions de clauses.



## Logique du premier ordre et théories

Revenons à l'exemple initial :

$$a \geq 0 \wedge b \geq 0 \implies a + b \geq b + c$$

Ici, les variables  $(a, b, c)$  sont des entiers, et on utilise des fonctions  $(+)$  et des prédicats  $(\geq)$ . C'est une **formule de la logique du premier ordre (FOL)**.

### Syntaxe FOL

- **Variables** :  $Var = \{a, b, c, \dots\}$ .
- **Domaine**  $D$  : un ensemble de valeurs (ex:  $\mathbb{Z}$ ).
- **Fonctions** :  $\mathbf{F} = \{f : D^k \rightarrow D\}$  (inclut les constantes,  $k = 0$ ).
- **Prédicats** :  $\mathbf{P} = \{P : D^k \rightarrow \{true, false\}\}$ .

### Termes :

- Une variable est un terme.
- Si  $t_1, \dots, t_k$  sont des termes et  $f \in \mathbf{F}$  d'arité  $k$ , alors  $f(t_1, \dots, t_k)$  est un terme.

### Atomes :

- $true$  et  $false$  sont des atomes.
- Si  $t_1, \dots, t_k$  sont des termes et  $p \in \mathbf{P}$  d'arité  $k$ , alors  $p(t_1, \dots, t_k)$  est un atome.

### Formules :

- Un atome est une formule.
- Si  $f_1, f_2$  sont des formules, alors  $\neg f_1, f_1 \wedge f_2, f_1 \vee f_2, f_1 \implies f_2$  sont des formules.
- Si  $f$  est une formule et  $x$  une variable, alors  $\exists x f$  et  $\forall x f$  sont des formules.

### Théories

Une **théorie**  $\mathbf{T}$  est un ensemble de formules FOL qui axiomatise un domaine particulier (ex: les entiers, les tableaux). Une formule  $f$  est **T-valide** si elle est conséquence de  $\mathbf{T}$ , i.e., pour toute interprétation  $I$  qui satisfait  $\mathbf{T}$  ( $I \models \mathbf{T}$ ), on a  $I \models f$ .

### Exemple : Théorie des entiers (Arithmétique linéaire)

- Fonctions :  $+$ ,  $-$ , constantes.
- Prédicats :  $\geq$ ,  $\leq$ ,  $=$ .
- Axiomes : propriétés de l'addition, de l'ordre, etc.

### Exemple : Théorie des tableaux

- Domaines : indices, valeurs, tableaux.
- Fonctions :  $select(a, i)$  (lire  $a[i]$ ),  $store(a, i, v)$  (écrire  $v$  en  $i$ ).
- Axiomes :  $\forall a, i, v, select(store(a, i, v), i) = v$ .

### Exemple : Égalité avec fonctions non interprétées (EUF)

- Prédicat :  $\approx$  (égalité).
- Axiomes d'équivalence : réflexivité, symétrie, transitivité.
- **Congruence** : pour toute fonction  $f$  et prédicat  $p$  :
  - $\forall x, y, x \approx y \implies f(x) \approx f(y)$
  - $\forall x_1, y_1, x_2, y_2, x_1 \approx y_1 \wedge x_2 \approx y_2 \implies f(x_1, x_2) \approx f(y_1, y_2)$
  - $\forall x, y, x \approx y \implies (p(x) \iff p(y))$

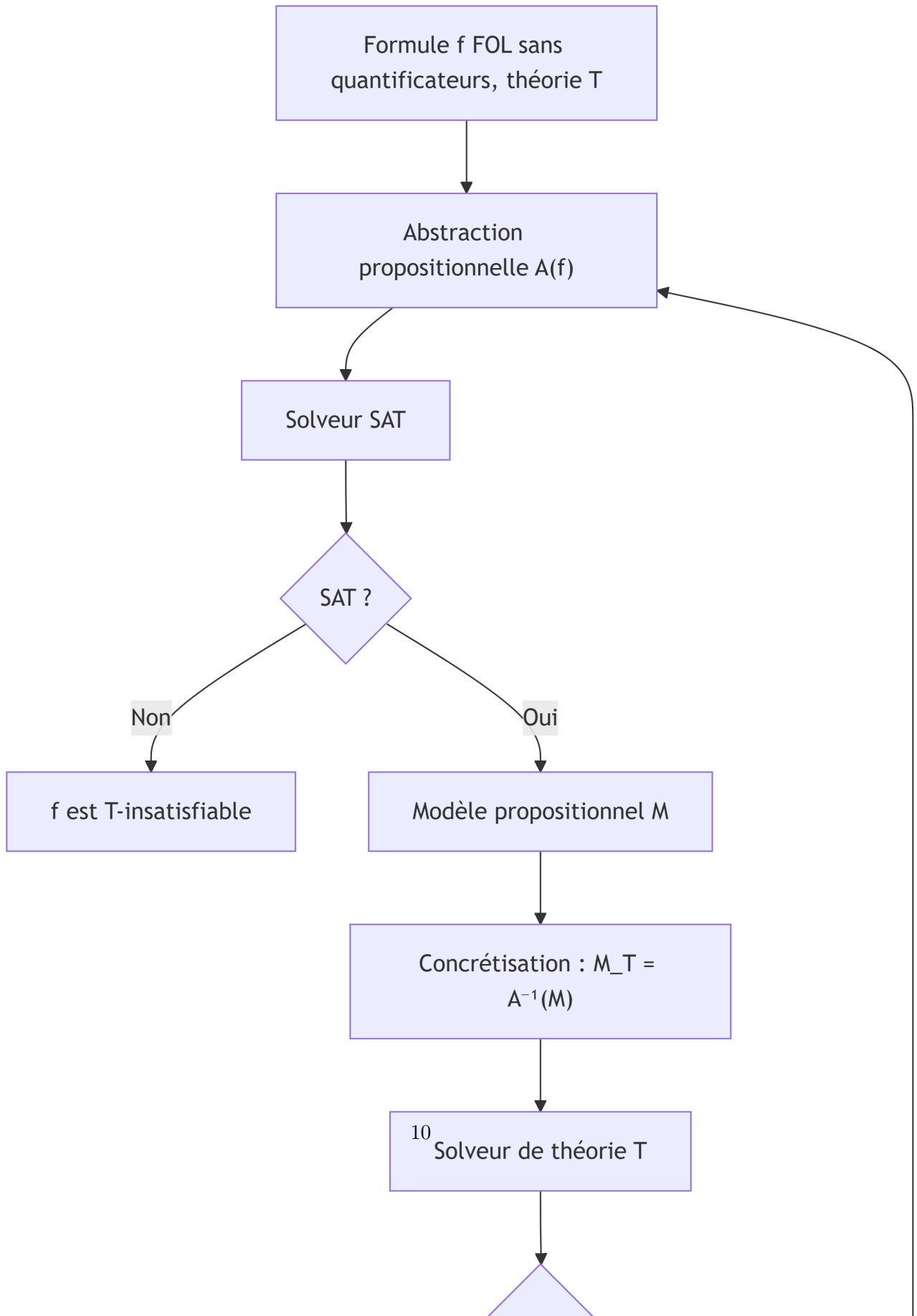
### SMT Solving : combiner SAT et théories

L'idée principale : utiliser un solveur SAT pour la structure logique propositionnelle, et des solveurs spécialisés pour les théories.





## Algorithme DPLL(T)



**Abstraction propositionnelle**  $A(f)$  : on remplace chaque atome de  $f$  par une nouvelle variable propositionnelle.

**Exemple :**

$$f = (a + b \leq c \wedge a > 0 \implies b + c < a) \vee a \leq 0$$

- $p_1 \leftrightarrow a + b \leq c$
- $p_2 \leftrightarrow a > 0$
- $p_3 \leftrightarrow b + c < a$
- $A(f) = (p_1 \wedge p_2 \implies p_3) \vee \neg p_2$

Si le solveur SAT trouve un modèle  $M$  (affectation des  $p_i$ ), on le **concrétise** en  $M_T = A^{-1}(M)$  (on remplace les  $p_i$  par les atomes correspondants). On vérifie si  $M_T$  est T-satisfiable à l'aide du solveur de théorie.

- Si oui,  $M_T$  est un modèle de  $f$ .
- Si non, on ajoute une **clause de refus**  $\neg M$  (la négation de l'affectation des  $p_i$ ) à  $A(f)$  pour éviter le même modèle incorrect, et on itère.

### Exemple complet avec la théorie EUF

Soit la formule  $P$  :

$$g(a) = c \wedge (f(g(a)) \neq f(c) \vee g(a) = d) \wedge c \neq d$$

Abstraction propositionnelle :

- $p_1 \leftrightarrow g(a) = c$
- $p_2 \leftrightarrow f(g(a)) = f(c)$
- $p_3 \leftrightarrow g(a) = d$
- $p_4 \leftrightarrow c = d$
- $A(P) = p_1 \wedge (\neg p_2 \vee p_3) \wedge \neg p_4$

Le solveur SAT peut trouver le modèle  $M = p_1 \wedge \neg p_2 \wedge \neg p_4$ .

Concrétisation :  $M_T = g(a) = c \wedge f(g(a)) \neq f(c) \wedge c \neq d$ .

Le solveur EUF détecte que  $g(a) = c$  implique  $f(g(a)) = f(c)$  (congruence). Donc  $M_T$  est EUF-insatisfiable.

On ajoute la clause de refus :  $\neg M = \neg p_1 \vee p_2 \vee p_4$ .

La nouvelle formule SAT est :  $p_1 \wedge (\neg p_2 \vee p_3) \wedge \neg p_4 \wedge (\neg p_1 \vee p_2 \vee p_4)$ .

Le solveur SAT peut maintenant trouver un autre modèle, jusqu'à ce qu'une solution T-satisfiable soit trouvée ou que la formule SAT devienne insatisfiable (ce qui prouve que  $P$  est T-insatisfiable).

## Formules FOL avec quantificateurs

### Définitions

- **Formule universelle** : de la forme  $\forall x_1, \dots, x_k \phi$ , où  $\phi$  est sans quantificateur.
- **Formule close** : toute variable est liée par un quantificateur.
- **Instance close** : pour une formule sans quantificateur  $\phi$ , une instance close est obtenue par une substitution  $\theta$  qui remplace chaque variable par un terme clos (sans variable libre). On note  $\theta(\phi)$ .

### Théorème de Herbrand (1930)

Un ensemble de formules FOL sans quantificateur est insatisfiable **ssi** il existe un ensemble fini d'instances closes qui est insatisfiable.

Cela permet de réduire l'insatisfiabilité d'une formule universelle close à l'insatisfiabilité d'un ensemble fini de formules propositionnelles (en instanciant les variables par des termes du domaine).

**Algorithme semi-décidable (TestSat) :**

```
F := { }  
tant que F est satisfiable faire  
    choisir une substitution telle que ( ) soit une instance close  
    F := F ∪ { ( ) }  
fin tant que  
retourner " est insatisfiable"
```

Si  $\phi$  est satisfiable, la boucle peut ne jamais terminer. Des techniques d'**instanciation guidée** (trigger, E-matching) sont utilisées en pratique.

### Exemple

Considérons le syllogisme :

- Tous les humains sont mortels.
- Tous les Grecs sont humains.
- Donc tous les Grecs sont mortels.

Formellement, on veut prouver que la formule suivante est valide :

$$(\forall x, H(x) \implies M(x)) \wedge (\forall x, G(x) \implies H(x)) \implies (\forall x, G(x) \implies M(x))$$

Par réfutation, on considère sa négation :

$$(\forall x, H(x) \implies M(x)) \wedge (\forall x, G(x) \implies H(x)) \wedge (\exists x, G(x) \wedge \neg M(x))$$

On la transforme en formule universelle close (étape par étape) :

1. **Skolémisation** : remplacer  $\exists x$  par une constante fraîche  $s$ .

$$(\forall x, H(x) \implies M(x)) \wedge (\forall x, G(x) \implies H(x)) \wedge (G(s) \wedge \neg M(s))$$

2. **Mettre sous forme prénexe** (tous les quantificateurs au début) :

$$\forall x, y, (H(x) \implies M(x)) \wedge (G(y) \implies H(y)) \wedge G(s) \wedge \neg M(s)$$

3. **Forme sans quantificateur interne** (on peut supprimer les quantificateurs universels pour l'instant, car on va instancier) :

$$\phi = (H(x) \implies M(x)) \wedge (G(y) \implies H(y)) \wedge G(s) \wedge \neg M(s)$$

Maintenant, on applique Herbrand. L'univers de Herbrand est constitué de la constante  $s$ . On prend l'instance close en remplaçant  $x$  et  $y$  par  $s$  :

$$\theta(\phi) = (H(s) \implies M(s)) \wedge (G(s) \implies H(s)) \wedge G(s) \wedge \neg M(s)$$

Cette formule propositionnelle est-elle satisfiable ?

- De  $G(s)$  et  $G(s) \implies H(s)$ , on déduit  $H(s)$ .
- De  $H(s)$  et  $H(s) \implies M(s)$ , on déduit  $M(s)$ .
- Mais on a  $\neg M(s)$ . Contradiction.

Donc  $\theta(\phi)$  est insatisfiable, ce qui prouve que la formule originale est valide.

## Transformation en formule universelle close

Pour appliquer Herbrand, on doit souvent transformer une formule FOL arbitraire en une formule universelle close qui préserve l'insatisfiabilité.

**Procédure :**

1. Éliminer  $\implies$  et  $\iff$  en utilisant  $\neg, \wedge, \vee$ .
2. Renommer les variables pour que chaque quantificateur lie une variable fraîche.
3. **Skolémisation** : remplacer chaque variable existentielle  $\exists y$  par un terme  $f(x_1, \dots, x_k)$  où  $f$  est un nouveau symbole de fonction (une constante si  $k = 0$ ) et  $x_1, \dots, x_k$  sont les variables universelles qui englobent  $\exists y$ .
4. Déplacer tous les quantificateurs universels en tête (forme prénexe).

**Exemple de skolémisation :**

- $\forall x, \exists y, P(x, y)$  devient  $\forall x, P(x, f(x))$  (avec  $f$  frais).
- $\exists y, Q(y)$  devient  $Q(c)$  (avec  $c$  frais).
- $\forall x, y, \exists z, R(x, y, z)$  devient  $\forall x, y, R(x, y, g(x, y))$  (avec  $g$  frais).

Cette transformation préserve l'insatisfiabilité, ce qui suffit pour une preuve par réfutation.

## Conclusion

Les solveurs SMT combinent la puissance des solveurs SAT modernes (CDCL) avec des procédures de décision pour diverses théories (EUF, arithmétique, tableaux, etc.).

Pour les formules avec quantificateurs, des techniques d'instanciation basées sur le théorème de Herbrand sont utilisées, souvent guidées par des triggers et l'E-matching.

Ces méthodes sont au cœur des outils de vérification déductive comme Why3, qui génèrent des obligations de preuve déchargées sur des solveurs SMT.